

LeafOS Inference Throughput Architecture

CPU Executive and RTX 4070 GPU Worker

LeafOS Project

Draft — 2026

Abstract

This paper records the initial throughput model and the evolving measurement protocol for a continual LeafOS inference loop. It separates CPU-resident executive work from GPU-resident worker work and explains why persistent state can increase useful throughput without increasing raw token generation speed. Historical rates are retained as model-specific observations, not hardware constants. Current measurements use `llama.cpp` with Vulkan and must record the model artifact, quantization, runtime build, placement, thread count, workload, thermal state, memory pressure, and competing processes.

1 Historical Initial CPU Executive Run

The earliest surviving record contains three CPU decode samples, a prompt-processing rate, and a rounded 7.37 GB weight footprint. It does *not* preserve the model path or hash, quantization name, `llama.cpp` build, backend, thread count, prompt and generation lengths, power mode, temperatures, memory pressure, or competing-process inventory. Consequently, this section is a dated baseline record rather than a current executive specification.

A later raw benchmark identifies the installed Gemma4 Opus Q4 artifact with an internal model size of 7,365,558,464 bytes, which rounds to the paper’s 7.37 GB footprint. It is therefore a plausible reconstruction candidate. That size agreement does not prove that it produced the earlier 3.90 tokens/s samples. The expanded rerun labels this association as unverified.

The recorded CPU decode samples are

$$3.81, \quad 3.96, \quad 3.92 \text{ tokens/s.}$$

Their mean is

$$\bar{r}_{\text{CPU}} = 3.897 \text{ tokens/s,}$$

and the sample standard deviation is approximately

$$\sigma \approx 0.078 \text{ tokens/s.}$$

The coefficient of variation is therefore

$$\text{CV} = \frac{0.078}{3.897} \times 100\% \approx 2.0\%.$$

The low variation says that the three samples were internally consistent under their original, unknown conditions. It does not establish stability across models, runtime builds, thermals, thread counts, or background workloads. The historical rounded rate was

$$r_{\text{brain}} = 3.9 \text{ tokens/s.}$$

Table 1 preserves the original arithmetic projection. These values are not measured sustained runs and exclude prompt processing, provider latency, tools, validation, checkpoints, retries, pauses, and thermal or process-state changes.

To make the amount of local work financially legible, let p_O be a comparison price in dollars per million generated cloud tokens. The gross cloud-equivalent value of N_O tokens generated locally is

$$V_{\text{local}} = \frac{N_O p_O}{1,000,000}.$$

The following early tables use $p_O = \$10$ per million generated tokens as an editable illustration, not as a claim about one provider or model. This is gross avoided output spend, not net savings: it excludes cloud input and cache charges as well as local electricity, cooling, storage, and hardware depreciation.

Table 1: Historical decode-only arithmetic and gross cloud-equivalent value at $p_O = \$10/\text{M}$.

Runtime	Local output	Cloud-equivalent value
1 hour	14,040 tokens	\$0.14
8 hours	112,320 tokens	\$1.12
12 hours	168,480 tokens	\$1.68
24 hours	336,960 tokens	\$3.37

For a generated work order of length L_O , the old decode-only upper bound was

$$\lambda_O = \frac{14,040}{L_O} \text{ orders/hour.}$$

Table 2: Historical decode-only upper bounds with gross cloud-equivalent value per local order at $p_O = \$10/\text{M}$.

Generated size	Upper bound/hour	Value per local order
300 tokens	46.8	\$0.003
500 tokens	28.1	\$0.005
800 tokens	17.6	\$0.008
1,500 tokens	9.4	\$0.015

The historical record also gives $r_{\text{prefill}} = 39.61$ tokens/s. Its simplified cycle estimate for prompt length L_P and generated length L_O was

$$t_{\text{brain}} = \frac{L_P}{39.61} + \frac{L_O}{3.90}.$$

For a 2,000-token state prompt and a 500-token work order,

$$t_{\text{brain}} = \frac{2000}{39.61} + \frac{500}{3.90} \approx 179 \text{ s.}$$

The 179-second result is therefore an inference-only example under the initial run’s unknown conditions, not an end-to-end work-order service level.

A more robust capacity model is

$$t_{\text{order}} = \frac{L_P}{r_{\text{prefill}}} + \frac{L_O}{r_{\text{decode}}} + t_{\text{provider}} + t_{\text{tools}} + t_{\text{validation}} \\ + t_{\text{checkpoint}} + t_{\text{retry}},$$

with

$$\lambda_{\text{observed}} \leq \frac{3600}{t_{\text{order}}}.$$

Each term and both inference rates must be measured for the model and operating conditions being reported.

1.1 Preserved 2026-07-17 Comparison Run

A later run has substantially better provenance. Its raw JSON records `llama.cpp` build 9957 at commit `c4ae9a88f`, Vulkan, an Intel i9-13900K, an RTX 4070, 24 CPU threads, batch 2048, micro-batch 512, F16 key and value caches, memory mapping, a 512-token prompt, 128 generated tokens, three repetitions, and no warm-up. Table 3 preserves the CPU-only results.

Table 3: Preserved model-specific CPU results from 2026-07-17.

Model artifact	Internal size	Prefill	Decode
Gemma4 Opus Q4	7.366 GB	326.47 tokens/s	4.051 tokens/s
Qwen3.5 14B-A3B MXFP4 MoE	8.830 GB	253.84 tokens/s	6.269 tokens/s
Qwopus3.5 9B Q4	5.769 GB	406.91 tokens/s	4.365 tokens/s

The same run measured the Qwen3.5 14B-A3B artifact at 6.259 tokens/s with zero GPU layers, 9.917 tokens/s with 16 GPU layers, and 107.550 tokens/s with full Vulkan offload. These are stronger evidence than the paper’s original GPU estimates, but they still describe one model, runtime build, workload, and unrecorded host/thermal state. The new matrix retains this protocol as a comparison point and adds thread, workload, cache, telemetry, idle-state, and drift controls.

2 RTX 4070 Bandwidth Model

For the historical model-specific calculation, let the quantized model footprint be $M_{\text{weights}} = 7.37$ GB. Pairing that footprint with the recorded CPU result implies an effective-bandwidth estimate of

$$B_{\text{CPU,eff}} \approx 7.37 \times 3.90 = 28.74 \text{ GB/s}.$$

Assuming an RTX 4070 VRAM bandwidth of approximately $B_{\text{VRAM}} = 504 \text{ GB/s}$ and ignoring all overhead, the theoretical decode ceiling is

$$r_{\text{ideal}} = \frac{504}{7.37} = 68.4 \text{ tokens/s}.$$

A 66 tokens/s target would require

$$B_{\text{required}} = 66 \times 7.37 = 486.42 \text{ GB/s},$$

or a bandwidth efficiency of

$$\eta_{\text{required}} = \frac{486.42}{504} = 96.5\%.$$

This is near the theoretical ceiling before accounting for quantized matrix-kernel overhead, KV-cache reads, attention, sampling, synchronization, and Vulkan dispatch overhead.

A more realistic decode model is

$$r_{\text{GPU}} \approx \frac{\eta B_{\text{VRAM}}}{M_{\text{weights}} + D_{\text{KV}} + D_{\text{overhead}}},$$

where η is achieved memory-bandwidth efficiency, D_{KV} is per-token KV-cache traffic, and D_{overhead} represents other memory traffic. Initially ignoring context overhead yields the estimates in Table 4.

Table 4: Model-specific decode rate implied by effective VRAM bandwidth alone for a 7.37 GB footprint.

Effective bandwidth	Expected decode
65%	44.5 tokens/s
75%	51.3 tokens/s
85%	58.1 tokens/s
90%	61.5 tokens/s
96.5%	66.0 tokens/s

The initial paper proposed the following model-specific GPU acceptance range:

$$50 \leq r_{\text{decode}} \leq 60 \text{ tokens/s},$$

with

$$r_{\text{stretch}} = 66 \text{ tokens/s}.$$

A Vulkan result of only 20–30 tokens/s for this artifact should trigger investigation of incomplete offload, excessive context, cache placement, thermal or process interference, and unsuitable kernels before concluding that the hardware is inadequate. It must not be generalized to a different dense or MoE model by model name alone.

3 Expected Acceleration

Relative to the historical 3.90 tokens/s CPU observation, the illustrative GPU worker speedup is

$$S = \frac{r_{\text{GPU}}}{3.90}.$$

Table 5: Illustrative GPU acceleration relative to the historical CPU observation.

GPU result	Speedup
45 tokens/s	11.5×
50 tokens/s	12.8×
55 tokens/s	14.1×
60 tokens/s	15.4×
66 tokens/s	16.9×

4 Full Offload Requirement

A 7.37 GB model nominally leaves

$$12 - 7.37 = 4.63 \text{ GB}$$

of a 12 GB VRAM budget for KV cache, compute buffers, and runtime state. This makes full model offload the primary implementation requirement.

If fraction f of model work is GPU-resident, and the remaining fraction is serially CPU-resident, the combined rate is modeled by

$$r_{\text{combined}} = \left(\frac{f}{r_G} + \frac{1-f}{r_C} \right)^{-1}.$$

Using $r_G = 55$ tokens/s and $r_C = 3.90$ tokens/s produces Table 6.

Table 6: Effect of incomplete GPU offload.

GPU-resident work	Combined result
90%	23.8 tokens/s
95%	34.3 tokens/s
99%	49.5 tokens/s
100%	55.0 tokens/s

Even a small serial CPU remainder materially reduces throughput. Full offload is therefore not merely an optimization; it is required to approach the worker target.

5 Executive–Worker Scheduling

The CPU executive does not need to equal the GPU worker’s raw generation rate. It needs to produce durable work orders more quickly than the worker consumes them. If the GPU worker requires 10–30 minutes to execute, test, and report a task, then

$$t_{\text{brain}} < t_{\text{worker}}.$$

The scheduling condition is

$$\frac{L_P}{r_{\text{prefill}}} + \frac{L_O}{r_{\text{brain}}} \leq t_{\text{worker}}.$$

Whether this condition is satisfied must be measured with the selected executive model and complete work-order timings. The CPU role remains persistent planning and task issuance; the GPU role remains interactive implementation and execution assistance.

6 Persistent State and Useful Throughput

Persistence does not directly raise raw GPU token generation speed. It raises useful throughput by avoiding repeated prompt processing, lost state, and regenerated work.

Using 55 tokens/s as an illustrative worker rate, without persistence a cycle is approximated by

$$T_{\text{cycle}} = \frac{C_{\text{full}}}{r_{\text{prefill}}} + \frac{O}{55} + T_{\text{redo}}.$$

With persistent KV cache and compact state, it becomes

$$T_{\text{cycle}} = \frac{\Delta C}{r_{\text{prefill}}} + \frac{O}{55} + T_{\text{checkpoint}},$$

where $\Delta C \ll C_{\text{full}}$.

If d is the fraction of generated worker output lost to repetition or state failure, useful throughput is

$$r_{\text{useful}} = 55(1 - d).$$

At 30% waste,

$$55(0.70) = 38.5 \text{ useful tokens/s.}$$

At 5% waste with persistent LeafOS state,

$$55(0.95) = 52.25 \text{ useful tokens/s.}$$

The resulting useful-throughput improvement is

$$\frac{52.25}{38.5} \approx 1.36,$$

or approximately 36%, with no change to the GPU’s raw decode speed.

7 Operating Model and Validation Criteria

The current operational equation is parameterized rather than fixed:

$$\begin{aligned} & r_{\text{executive}}(m_e, c_e) + r_{\text{worker}}(m_w, c_w) \\ & \quad + \text{persistent KV/state/checkpoints} \\ & = \text{continuous LeafOS loop.} \end{aligned}$$

Here m_e and m_w identify exact model artifacts, while c_e and c_w contain the runtime, placement, workload, thermal, power, memory, and competing-process conditions.

The immediate validation sequence is:

1. Confirm that Windows and the inference runtime recognize the RTX 4070 and Vulkan back-end.
2. Require an idle preflight and record the model artifact, runtime build hashes, prompt and generation lengths, repetitions, CPU threads, GPU layers, KV types, VRAM, temperatures, power, host load, and process inventory.
3. Sweep CPU thread counts per model, because hybrid-core scheduling can change the executive result materially.
4. Compare CPU-only, partial-offload, and full-offload placements using the same model and workload.
5. Treat 50–60 tokens/s and the 66 tokens/s stretch value only as historical expectations for the 7.37 GB calculation until the clean matrix supplies measured replacements.
6. Repeat the full-GPU cell at matrix end to measure thermal and host-state drift.
7. Retain compact executive state, checkpoints, and KV cache to maximize useful work rather than raw benchmark output alone.

8 Historical GLM-5.2 Overnight Evaluation (Deprecated)

This archived evaluation asked whether FlowerOS could sustain useful unattended inference with GLM-5.2 Q4 under a constrained memory hierarchy. GLM-5.2 was deprecated for local use on 2026-07-21 because it exceeds the practical envelope of the reference home workstation. The proposed deployment had 467 GB of stored weights, 256 routed experts with 8 active per token, a 48 GB RAM hot cache, a 12 GB RTX 4070 selective-offload budget, and fast NVMe memory-mapped access. These values defined a proposed test configuration; they were not a measured throughput claim or an implemented LeafOS SSD tier manager.

The memory and execution path is

NVMe mmap streaming → 48 GB RAM hot cache
→ 12 GB RTX 4070 selective offload
→ 8 active routed experts.

Because only a subset of experts is active per token, sustained performance depends on expert locality, page-cache residency, storage latency, and the placement policy as well as nominal model size.

8.1 Generation-Rate Gates

Table 7 defines the initial interpretation gates for generation throughput.

Table 7: GLM-5.2 Q4 overnight inference benchmark gates.

Result	Interpretation
Less than 0.10 tokens/s	Technically runs, but operationally useless
0.10–0.29 tokens/s	Experimental persistence test
0.30 tokens/s	Minimum success
0.40 tokens/s	Practical overnight target
0.66 tokens/s	FlowerOS optimization success
At least 1.0 tokens/s	Exceptional result

8.2 Comparison Layers

Each test must compare three layers using the same model file, prompt workload, context policy, and measurement window:

1. **Raw baseline:** llama.cpp with minimal configuration.
2. **FlowerOS optimized:** CPU MoE placement, mmap, GPU auto-fit, I/O priority, cache warming, and thread tuning.
3. **LeafOS productive:** compaction, persistent state, code sandboxes, testing, and seven-hour output measurement.

This progression separates the raw runtime cost of sparse-model inference from platform optimizations and from productive, validated work.

8.3 Measurement Protocol

Every run must record the following metrics:

- model load time and time to first token;
- prompt-processing and generation rates;
- cold-cache and warm-cache generation rates;
- NVMe read bandwidth and latency;
- major and minor page faults, plus RAM page-cache residency;
- GPU utilization and VRAM usage;
- CPU utilization and thermals;
- tokens per watt-hour;
- validated patches per seven hours, accepted versus rejected changes; and
- recovery behavior after interruption.

The primary outcome is not raw token rate. It is productive throughput:

$$\text{Productivity} = \frac{\text{validated useful changes}}{\text{seven-hour run}}.$$

A model operating at 0.35 tokens/s can be worthwhile if compaction and persistent state prevent repeated context reconstruction, while sandboxing and tests allow it to produce validated patches steadily through the night.

9 Current Medium-MoE Qualification Policy

The current operator decision favors mixture-of-experts candidates with 47–156 billion total parameters. This range is a selection band, not proof of host fit. Each candidate must record total and active parameters, expert topology, quantized GGUF identity, pinned revision, hash, context, explicit llama.cpp command, and pre-run acceptance thresholds.

The measured Qwen3.5 14B A3B baseline produced 6.259 tokens/s on CPU, 9.917 tokens/s with partial Vulkan offload, and 107.550 tokens/s with full Vulkan offload. That result supports measuring the complete placement path, but it is outside the new total-parameter band and cannot be extrapolated into a medium-MoE throughput promise. Resident trials also showed that the current Q3 route preserved workstation headroom where larger routes reached RAM pressure.

Qualification therefore uses a 4 minute cold-cache run, an 8 minute warm-cache run, and a 64 minute resident-foreground run. It records provider survival, restart count, out-of-memory events, RAM and VRAM pressure, foreground responsiveness, token rates, and validated useful changes per hour. One report cannot establish promotion eligibility. The portfolio qualifier must verify all three passing reports before operator review; promotion is never automatic.

The storage contract is deliberately narrow: llama.cpp may memory-map a local GGUF and the operating system may satisfy pages from SSD. LeafOS does not currently implement expert pinning or managed SSD/RAM/VRAM tiering. The machine-readable authority is `config/medium_moe_policy.json`, with candidate shape in `schemas/leafos.medium-moe-candidate.v1.schema.json`.